

Technische Universität Clausthal

Department of Informatics

Benchmarking LLM-Generated Web Interfaces

A Reproducible Multi-Metric Leaderboard Framework

Master's Thesis

Xinyang Xie

June 11, 2026

Contents

1	Introduction	1
2	Background and Related Work	2
2.1	Research Gap	2
3	Benchmark Design and Methodology	5
3.1	Research Questions	5
3.2	Overall Benchmark Design	6
3.3	Benchmark Item and Generated Artifact	7
3.4	Dataset Tracks	8
3.4.1	Track A: Baseline or Sanity Check	8
3.4.2	Track B: Generated Web Interfaces	8
3.5	Generation Protocol	9
3.6	Rendering and Screenshot Protocol	9
3.7	Metric Categories	10
3.7.1	Static Technical Score	10
3.7.2	Static Visual Score	10
3.7.3	Accessibility Score	11
3.7.4	Dynamic Score	12
3.7.5	Efficiency Score	12
3.7.6	Optional Overall Score	12
3.8	Dynamic Validation Strategy	13
3.9	Failure Taxonomy	13
3.10	Reliability Audit	14
3.11	Research Question – Method Map	15
A	Prompt Templates and Experimental Metadata	18
A.1	Inclusion Rule	18
A.2	Track A Pairwise Judgement Prompts	18
A.2.1	TA-PAIR-ZS-v1: Zero-shot Pairwise Visual Design Prompt . . .	18

A.2.2	TA-PAIR-RUBRIC-v1: Rubric-guided Pairwise Visual Design Prompt	19
A.3	Track B GUI Generation Prompts	20
A.3.1	Track B Generation Prompt Version History	20
A.3.2	TB-GEN-v9: Per-Item Workflow-Label Generation Prompt . . .	21
A.3.3	TB-GEN-v14: Visual-Grounded Workflow-Control Diagnostic Prompt	23
A.3.4	TB-GEN-v15: Stricter Route and Click-Target Diagnostic Prompt	25
A.3.5	TB-GEN-v16: Compact-Input Visual Workflow Candidate . . .	27
A.4	Track B Static-Visual Judge and Human Review Prompts	29
A.4.1	LB-JUDGE-v1: Base Static-Visual Checklist Prompt	29
A.4.2	LB-JUDGE-v1: Strict Variant	31
A.4.3	LB-JUDGE-v1: Strict Few-Shot Variant	31
A.4.4	HUMAN-VISUAL-v1: Human Review Checklist	32
A.5	Prompt Slots for Later Experiments	33

Chapter 1

Introduction

This chapter will introduce the motivation, research context, and contribution of the thesis. It is intentionally kept as a short placeholder so that the following draft chapters keep their intended numbering while the full introduction is developed.

Chapter 2

Background and Related Work

2.1 Research Gap

The preceding sections have reviewed GUI quality assessment, LLM-generated interfaces, LLM-as-a-judge methodology as an evaluation tool, and multimodal web agents. Table 2.1 places the most relevant prior works along seven dimensions that together characterise a comprehensive GUI quality evaluation framework. Taken together, three observations motivate the present thesis.

Existing website-generation benchmarks do not directly answer generator-leaderboard questions. UIClip [13] demonstrated that vision-language models can perform pairwise UI comparison, but it relies on models that predate the current generation of multimodal LLMs and limits evaluation to a binary preference. The LLM-as-a-judge literature [15, 6] has established scalable evaluation methods for text, yet those methods also introduce known bias and calibration concerns. In this thesis they are therefore treated as practical scoring instruments rather than as the main leaderboard target. Vision2Web [1] moves closer to this thesis by evaluating visual website development with both visual judging and agent-based verification, but its primary object is coding-agent website development rather than a thesis-specific, multi-metric leaderboard for comparing generated GUI submissions across static and dynamic metrics.

Requirement fidelity and rubric-based assessment are largely absent from GUI evaluation. Design2Code [11] evaluates how faithfully generated code reproduces a visual reference, but does not assess richer quality dimensions such as visual hierarchy, information clarity, or perceived usability. The Designer Feedback study [14] found that simple pairwise rankings among evaluators agreed at only around 50%, suggesting that coarse binary judgements are insufficient for reliable GUI assessment. A structured rubric grounded in established usability theory [9, 2] can provide finer-grained

and more reproducible quality signals.

Static visual metrics have rarely been related to dynamic interaction outcomes. VisualWebArena [3] and WebArena [16] evaluate multimodal agents on goal-directed web tasks, but do not assess the GUI quality of the interfaces themselves. How static quality scores relate to task completion, interaction feedback, and functional behaviour remains an open empirical question. Answering this question requires a benchmark that keeps generated interfaces executable so that static and dynamic evaluations can be performed on the same set of items.

The present thesis addresses all three gaps simultaneously by centring the benchmark on controlled generation of executable web interfaces, standardized artifact scoring, model-level aggregation, and a static-plus-dynamic leaderboard for comparing generator LLMs. UIClip-style pairwise judgement is retained only as a small baseline or sanity check. LLM-based scorers and human review are treated as scoring instruments and audit signals rather than as the main leaderboard target.

Table 2.1: Comparison of related work across seven evaluation dimensions. ✓ = fully addressed; △ = partially addressed; × = not addressed; — = not applicable to this line of work.

Work	Multi-dimensional rubric	Generated GUI quality	Bias and reliability analysis	Human reference labels	Requirement fidelity	Dynamic interaction eval	Benchmark artifact
UIClip [13]	×	×	×	✓	×	×	×
Designer Feedback [14]	×	✓	×	✓	△	×	×
Design2Code [11]	△	✓	×	✓	✓	×	✓
Vision2Web [1]	△	✓	△	△	✓	✓	✓
MT-Bench / Chatbot Arena [15]	△	—	△	✓	—	—	△
LLM-as-a-Judge survey [6]	✓	—	✓	—	—	—	×
Position Bias Study [10]	×	—	✓	✓	—	—	×
WebArena [16]	×	△	×	✓	△	✓	✓
VisualWebArena [3]	×	✓	×	✓	△	✓	✓
This thesis	✓	✓	△	△	✓	✓	✓

Chapter 3

Benchmark Design and Methodology

This chapter defines the benchmark and evaluation protocol used in this thesis. The central object being scored is the generated UI or Web app artifact. The leaderboard ranking target is the generator LLM: artifact-level scores are aggregated across a fixed benchmark item set into model-level leaderboard rows.

The chapter first states the research questions, then defines the benchmark object model, generation protocol, rendering protocol, metric categories, dynamic validation strategy, reliability audit, and the mapping from research questions to methods.

3.1 Research Questions

The thesis investigates how to evaluate LLM-generated web interfaces with a reproducible benchmark and leaderboard. It uses automated scoring, human review, and LLM-based judging as measurement instruments, but these instruments are not the main leaderboard target.

RQ1 – Benchmark and leaderboard design.

How can a reproducible benchmark and leaderboard be designed for evaluating LLM-generated web interfaces?

RQ1 defines the fixed benchmark dataset, benchmark item schema, generation protocol, generated artifact format, metadata requirements, and aggregation from artifact-level scores to generator-model leaderboard rows. Track A/UIClip-style material is retained only as a small baseline or sanity check, not as the main research question or primary contribution.

RQ2 – Metric operationalization and aggregation.

How can static technical, static visual, dynamic task-based, and efficiency metrics be operationalized and aggregated for generated web apps?

RQ2 defines the category scores used by the leaderboard: Static Technical Score, Static Visual Score, Dynamic Score, and Efficiency Score. An optional Overall Score may be reported, but category-specific rankings remain visible because different users may prioritize visual quality, functional correctness, or cost efficiency differently.

RQ3 – Static and dynamic quality relationship.

How are static quality scores related to dynamic task-success outcomes in LLM-generated web apps?

RQ3 evaluates the same generated artifacts with both static and dynamic signals. The analysis reports correlations and mismatch cases, such as visually strong interfaces that fail tasks or visually plain interfaces that complete required workflows reliably.

Reliability audit.

How stable and bias-sensitive are the LLM-based visual or rubric scores used in the evaluation pipeline?

The reliability audit supports the scoring protocol. It records repeated scoring consistency, order-swap sensitivity for pairwise comparisons when used, rating-scale sensitivity if feasible, judge-model variance, and invalid or refusal rates. It is not a separate main research question and does not make judge-model comparison the thesis target.

3.2 Overall Benchmark Design

The benchmark is based on fixed items and controlled generation. A generator model receives the same benchmark context and standardized prompt schema as other models in the same comparison batch. The model produces executable web artifacts, and those artifacts are scored with the same evaluation protocol.

Direct evaluation target. The generated UI/Web app artifact is scored for one benchmark item. Scores include technical coverage, visual quality, dynamic task behavior, and efficiency metadata.

Leaderboard ranking target. The generator LLM or generator model configuration is ranked. Artifact-level scores are aggregated across benchmark items into model-level rows.

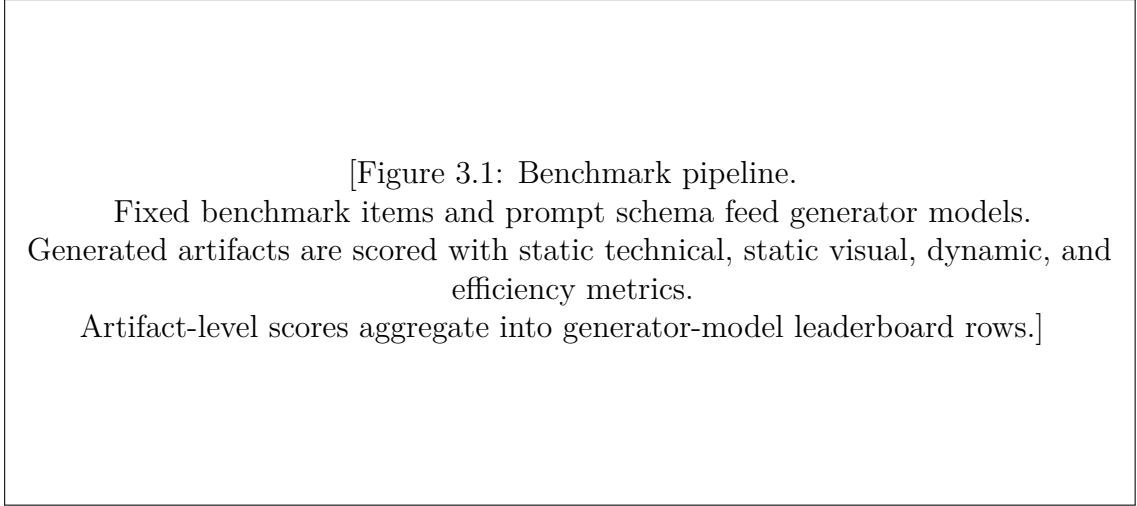


Figure 3.1: Benchmark and leaderboard pipeline. Generated artifacts are the direct scoring target; generator models are the leaderboard ranking target.

Primary mode. The main mode is New LLM mode: a generator model API/interface is added to the pipeline, the fixed benchmark set is run, and the result is a model row in the leaderboard.

Secondary mode. A single HTML file or zipped project may be scored as an artifact-level report, but this is secondary. It is not the main leaderboard basis unless the artifact was generated under the fixed benchmark protocol and includes complete model, prompt, item, timestamp, and generation metadata. For controlled thesis experiments, complete provider-parameter, token-usage, latency, and finish-reason metadata are also recorded. For a public artifact-scoring mode, the benchmark can accept a submitted executable artifact with minimal optional metadata and still return an artifact-level diagnostic report, but such reports are not used as controlled model-comparison evidence unless the missing metadata are supplied.

3.3 Benchmark Item and Generated Artifact

A benchmark item is the fixed test specification. It is not a generated output. Each item contains the information needed to generate and evaluate a web interface:

- natural-language requirement text;
- expected pages or routes;
- required elements, text, controls, forms, and selectors;
- dynamic tasks or workflows;
- validation rules;

- predefined pages or routes for static visual screenshots;
- metadata such as source, category, difficulty, and whether dynamic validation is required.

The recommended planning schema is:

```
benchmark_item/
  requirement.md
  pages.json
  elements.json
  tasks.json
  validation_rules.json
  visual_pages.json
  metadata.json
```

A generated artifact is the output produced by one generator model for one benchmark item. The minimal artifact is an executable web implementation plus generation metadata. The metadata records model identity, provider, prompt ID, timestamp, provider parameters, token usage, latency, cost, and finish reason.

3.4 Dataset Tracks

3.4.1 Track A: Baseline or Sanity Check

Track A uses a reduced UIClip-style screenshot baseline. Its purpose is to anchor the study in prior screenshot-based GUI evaluation and to provide a sanity check for pairwise judging. It is not the primary contribution and does not define the main leaderboard objective.

Track A can reuse existing UIClip preference labels for a small sample of screenshots. No new large annotation campaign is required for this baseline.

3.4.2 Track B: Generated Web Interfaces

Track B is the main benchmark track. It uses requirement-driven web interface generation tasks, primarily from a reduced Vision2Web Level 1/Level 2 subset. Level 3 full-stack tasks are excluded because they introduce backend state, deployment, and long-horizon engineering requirements that are outside the current benchmark scope.

The development prototype should begin with 2–5 benchmark web app items. If feasible, the thesis experiment may expand to 10–20 benchmark tasks. Expansion depends on generation cost, scoring cost, rendering reliability, and the amount of usable dynamic validation available in the selected items.

3.5 Generation Protocol

For direct generator comparisons, all models in a comparison batch receive the same benchmark item context and the same standardized prompt schema. Model-specific prompt tuning is not allowed within a batch because it would confound generator quality with prompt engineering.

The main input to the leaderboard pipeline is a generator model API/interface. For each model, the pipeline loads the fixed benchmark items, applies the standardized prompt, saves generated artifacts, records generation metadata, and runs the evaluation protocol.

The evaluation should avoid presenting any fixed truncating output-token cap as the final policy. If a provider requires a maximum output setting, it should be set high enough to allow complete outputs where feasible. Actual input tokens, output tokens, total tokens, finish reason, cost, and latency are recorded as efficiency and reproducibility data.

The current Track B generation protocol uses versioned prompt identifiers and keeps prompt wording separate from input-packing policy. The compact-input candidate, **TB-GEN-v16**, reduces repeated workflow context by passing a deduplicated route/evidence summary, separates exact visible-text controls from semantic visual targets, ranks local resource paths, and downscales prototype images for compact runs. The purpose is to reduce context and output pressure without silently changing the task. Prompt variants are not edited in place: material changes create a new prompt identifier, and failures are separated into capacity, provider, model instruction-following, gate, and dynamic validation causes before deciding whether a prompt revision is justified.

For OpenAI-compatible providers, the pipeline may omit `max_tokens` when the provider accepts that request, rather than imposing the script’s default cap. If a run still stops with a length-related finish reason after the client cap is omitted, the failure is treated as provider or model completion behavior under the fixed prompt, not as automatic evidence that the prompt must be changed.

Prompts that affect thesis results, metric computation, benchmark protocol, human annotation, output schemas, or reproducibility are versioned and preserved according to the prompt preservation policy.

3.6 Rendering and Screenshot Protocol

Generated artifacts are rendered locally with a controlled browser environment. Rendering produces the evidence used by later scoring modules.

For static visual scoring, the pipeline visits only the predefined pages or routes specified by the benchmark item and captures standardized screenshots. The browser

engine, viewport, zoom level, wait rule, and screenshot mode are kept fixed across artifacts. These screenshots are the input to static visual evaluation.

Agent trace screenshots are not used as the main static visual scoring input. They may be stored for debugging and dynamic failure analysis only, because agent traces can vary in length, path, and content across generated apps.

Rendering failures such as blank pages, fatal JavaScript errors that prevent visible content, or missing entry files are recorded as artifact-level failures.

3.7 Metric Categories

3.7.1 Static Technical Score

Static technical metrics evaluate structure and requirement coverage without executing a user task. Candidate submetrics include HTML completeness, required element coverage, required text coverage, route declaration coverage, link target declaration, form field coverage, selector or ID coverage, and static requirement fidelity. Rule-based accessibility is reported separately as the Accessibility Score (Section 3.7) because it is computed by an automated tool rather than by structural heuristics.

Static technical checks can verify that a required button exists and declares a target. They do not prove that clicking the button completes a task; that is a dynamic metric.

The current Track B gate treats complete HTML, non-truncated style and script blocks, defined click handlers, declared route identifiers, and working workflow-required controls as hard checks. Exact visible-label matching is a hard check only for workflow actions that specify quoted UI labels; more semantic click descriptions are recorded as diagnostic warnings and are resolved by route or task validation.

3.7.2 Static Visual Score

Static visual metrics evaluate standardized predefined-route screenshots. To keep the score reproducible and to avoid injecting high judge variance into the ranking, the design follows two principles. First, anything with an objective rule or formula is not LLM-judged: contrast, for example, has a deterministic WCAG formula computable from the actual CSS colors and is therefore moved to the automated accessibility metrics rather than scored by a multimodal judge. Second, holistic aesthetic judgement is treated cautiously, because multimodal judges align poorly with human aesthetic ratings without calibration [8, 4]; overall aesthetic quality is consequently deferred to future work, or reported only as an optional, non-weighted diagnostic accompanied by human-correlation validation.

The multimodal judge therefore scores four objective-leaning visual dimensions: *layout and visual hierarchy*, *information organization and clarity*, *typography and readability*,

and *visual consistency*. These build on dimensions used by recent multimodal-LLM UI and web-development judges [7, 12] and on validated aesthetics inventories [8, 4]. Each dimension is scored by decomposing it into binary (yes/no) sub-questions answered by the judge from the screenshot, rather than by eliciting a raw Likert score; checklist decomposition yields higher inter-judge agreement and lower variance than holistic Likert ratings [5]. A dimension score is the fraction of passed sub-questions. Where a graded judgement is unavoidable, a narrow three-to-five level scale with explicit behavioral anchors is used rather than a wide scale, which is prone to central-tendency bias; a wide or verbal-label scale is reserved only as a deliberate condition when recording rating-scale sensitivity.

Page-level visual scores are aggregated into an artifact-level Static Visual Score, with each dimension first normalized to $[0, 1]$. Agent trace screenshots are excluded from the main static visual score.

3.7.3 Accessibility Score

Rule-based accessibility is evaluated by a free, local, deterministic tool (**axe-core** via Playwright, or Lighthouse) rather than by a multimodal judge. This is cheaper than judge-based scoring, more accurate, reproducible, and immune to judge bias, while still contributing to the leaderboard under the automated technical and accessibility category. Tool-checkable items include color contrast on the actual CSS colors, image alternative text, form-control labels, accessible button and link names, sensible heading order, declared document language, correct ARIA usage, and landmark regions.

The leaderboard accessibility score uses severity-weighted violation density rather than the raw pass ratio, because the pass ratio clustered too tightly in the development subset to discriminate artifacts. The frozen formula is:

```
weighted_violations =
    1 * minor + 2 * moderate + 4 * serious + 8 * critical

Accessibility Score =
    1 - min(1, weighted_violations / (pass_nodes + violation_nodes))
```

The weights are a recorded convention over **axe-core**’s ordinal impact levels. The raw pass ratio remains a diagnostic field, but it does not enter the leaderboard score. Because many simple content pages trigger only a narrow set of applicable accessibility rules, especially colour contrast, the scale-up item set should include forms, tables, and richer document structure where possible.

3.7.4 Dynamic Score

Dynamic metrics evaluate interaction behavior. A metric is dynamic when it requires clicking, typing, navigating, submitting, waiting for state changes, or validating task completion.

Candidate submetrics include task success rate, step success rate, route execution success, button functionality success, form completion success, state-change success, retry count, failure localization, and robustness or pass@k.

The current implementation separates the dynamic metric schema from the executor. Early pilot runs may use deterministic route simulation, while the browser workflow evaluator executes the same workflow cases in Chromium with Playwright, clicks generated controls, and validates visible destination content. A future implementation may replace this evaluator with a real computer-use agent, but the metric schema should remain stable: change the executor, not the metric schema.

3.7.5 Efficiency Score

Efficiency metrics track input tokens, output tokens, total tokens, generation cost, evaluation cost, generation latency, evaluation latency, and cost-performance trade-offs.

Efficiency is reported both as raw metadata and, when a reference is defined, as a normalized score. Cost-performance plots and Pareto frontiers are used to show whether a model is dominated by another model with both lower cost and higher quality.

3.7.6 Optional Overall Score

The initial optional composite score is:

```
Overall Score =  
0.25 * Static Technical Score  
+ 0.25 * Static Visual Score  
+ 0.35 * Dynamic Score  
+ 0.15 * Efficiency Score
```

This composite is a reporting convenience, and its weights are an un-justified hyper-parameter to be sensitivity-tested rather than a fixed final value. Category-specific scores remain visible and are interpreted separately. Because cost is primarily presented as a Pareto-plot axis, whether the Efficiency Score also enters the weighted composite is deferred to avoid double-counting cost.

3.8 Dynamic Validation Strategy

The thesis first implements workflow-based dynamic validation rather than heavy real-agent execution. This choice keeps the pipeline reproducible and avoids confounding generated-interface quality with the planning or grounding failures of a separate agent.

For Vision2Web-derived Level 2 items, dynamic tasks can be derived from workflow actions and validations. The evaluator checks whether the generated artifact supports the expected route, form, state, or content outcome. The current pipeline distinguishes deterministic route simulation from browser-workflow validation. Route simulation reads the generated artifact and checks route/content evidence without driving a real browser. Browser-workflow validation opens the generated HTML in Chromium, executes workflow clicks with Playwright, records route success, and validates visible destination content. Both are dynamic validation signals, but neither is presented as a full real-agent usability study.

The browser-workflow evaluator is currently used as a pilot implementation of the dynamic layer. Its pilot results are used to validate the protocol and identify workflow-design issues, not as final thesis results until the benchmark item set, workflow normalization, and reporting rules are fixed. In particular, workflow cases that depend on nested navigation must be normalized so that each independently scored case contains the prerequisite steps needed to expose its target control.

Real-agent execution remains an optional future upgrade or small cross-check after the main pipeline is stable. If added later, the same task success and failure taxonomy should be reused so that only the executor changes.

3.9 Failure Taxonomy

Failures are recorded so that leaderboard scores remain interpretable:

- **Provider failure:** no artifact because the provider times out, rejects the request, returns an HTTP/API error, or otherwise fails before a usable completion is produced.
- **Completion or truncation failure:** the model returns an incomplete or unusable artifact, including length-related finish reasons or unclosed document, style, or script blocks.
- **Render failure:** artifact exists but cannot be rendered into visible content.
- **Static technical failure:** required structure, route, element, text, selector, handler, or route declaration evidence is missing.

- **Static visual failure:** standardized screenshot shows severe visual defects such as overlap, unreadable text, or unusable layout.
- **Dynamic route failure:** required clicks or actions do not reach the expected route or state.
- **Dynamic content-validation failure:** the expected route or state is reached, but the visible destination content does not satisfy the validation evidence.
- **Evaluator/tool failure:** scoring or validation tool fails for reasons unrelated to the generated artifact.

Evaluator or tool failures are reported separately from generated-artifact quality when possible.

3.10 Reliability Audit

LLM-based visual or rubric scoring can be affected by prompt sensitivity, position bias, scale effects, model variance, and invalid outputs. These issues are audited but are not the main thesis target.

The audit may include repeated scoring consistency, order-swapped pairwise comparisons where pairwise scoring is used, rating-scale sensitivity if feasible, judge model variance, and invalid or refusal rate. Audit results are reported as scoring reliability evidence and threats to validity.

For the static visual checklist, the reliability audit uses the same 16-item binary rubric for human annotation and LLM judging. The audit records raw item agreement, page-score correlation, chance-corrected agreement such as Cohen’s κ , invalid or unparsable outputs, and score degeneracy such as all-true or near-constant responses. A strict judge prompt and few-shot calibration examples may be compared against a base prompt, but this comparison audits the measurement instrument; it does not make judge-model ranking the main thesis objective.

The development validation also includes deliberately degraded screenshot variants to test whether visual judges can separate large quality gaps when the original artifact set is range-restricted. Mild perturbations are treated as a sensitivity check rather than as a guaranteed error oracle, because human review showed that small layout or spacing changes can be perceived as neutral or even better. Severe perturbations and weak-model artifacts provide the main low-quality evidence for full-range validation. Any final production-judge choice is therefore based on calibrated agreement, page-score correlation, behavior on weak artifacts, and documented failure modes, not on raw agreement alone.

Table 3.1: Research question – method mapping.

RQ	Question	Data	Method	Primary outputs
1	Benchmark and leaderboard design	Fixed benchmark items; generated artifacts	New LLM mode; artifact scoring; model-level aggregation	Benchmark schema; prompt protocol; leaderboard row format
2	Metric operationalization and aggregation	Generated artifacts; screen-shots; task validations; meta-data	Static technical, static visual, dynamic, and efficiency scoring	Category scores; optional Overall Score; cost-performance plots
3	Static and dynamic quality relationship	Same Track B artifacts scored statically and dynamically	Correlation analysis and mismatch case analysis	Static-dynamic correlations; qualitative divergence cases
	Audit Scoring reliability	Selected scoring outputs	Repeated scoring, order-swap checks, variance and invalid-rate logs	Reliability and bias audit results

3.11 Research Question – Method Map

Table 3.1 maps the research questions to data, methods, metrics, and outputs.

Bibliography

- [1] Zehai He, Wenyi Hong, Zhen Yang, et al. Vision2Web: A hierarchical benchmark for visual website development with agent verification. *arXiv preprint arXiv:2603.26648*, 2026.
- [2] International Organization for Standardization. ISO 9241-11:2018 — ergonomics of human-system interaction: Usability: Definitions and concepts. International Standard 9241-11, ISO, 2018.
- [3] Jing Yu Koh, Robert Lo, Lawrence Yunliang Chen, et al. VisualWebArena: Evaluating multimodal agents on realistic visual web tasks. In *Proc. ACL*, 2024.
- [4] Talia Lavie and Noam Tractinsky. Assessing dimensions of perceived visual aesthetics of web sites. *International Journal of Human-Computer Studies*, 60(3): 269–298, 2004.
- [5] Yukyung Lee et al. CheckEval: A reliable LLM-as-a-judge framework for evaluating text generation using checklists. *arXiv preprint arXiv:2403.18771*, 2024.
- [6] Dawei Li, Bohan Jiang, Liangjie Huang, et al. From generation to judgment: Opportunities and challenges of LLM-as-a-judge. In *Proc. EMNLP*, 2025.
- [7] MLLM as a UI Judge authors. MLLM as a UI judge: Benchmarking multimodal LLMs for predicting human perception of user interfaces. *arXiv preprint arXiv:2510.08783*, 2025.
- [8] Morten Moshagen and Meinold T. Thielsch. Facets of visual aesthetics. *International Journal of Human-Computer Studies*, 68(10):689–709, 2010.
- [9] Jakob Nielsen and Rolf Molich. Heuristic evaluation of user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, pages 249–256. ACM, 1990.
- [10] Lin Shi, Chiyu Ma, Wenhua Liang, et al. Judging the judges: A systematic study of position bias in LLM-as-a-judge. In *Proc. IJCNLP-AAACL*, 2025.

- [11] Chenglei Si, Yanzhe Zhang, Zhengyuan Yang, et al. Design2Code: How far are we from automating front-end engineering? In *Proc. ACL*, 2024.
- [12] WebDevJudge authors. WebDevJudge: Evaluating (m)LLMs as critiques for web development quality. *arXiv preprint arXiv:2510.18560*, 2025.
- [13] Jason Wu, Yi-Hao Peng, Xin Yue Amanda Li, et al. UIClip: A data-driven model for assessing user interface design. In *Proc. ACM UIST*, 2024.
- [14] Jason Wu, Amanda Swearngin, Arun Krishna Vajjala, et al. Improving user interface generation models from designer feedback. In *Proc. CHI*, 2026.
- [15] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *NeurIPS*, 2023.
- [16] Shuyan Zhou, Frank F. Xu, Hao Zhu, et al. WebArena: A realistic web environment for building autonomous agents. In *Proc. ICLR*, 2024.

Appendix A

Prompt Templates and Experimental Metadata

This appendix records prompts that materially affect the thesis methodology, experimental results, benchmark protocol, or reproducibility. Routine prompts used for coding, brainstorming, language polishing, or project management are not included unless their outputs become part of the experimental protocol.

A.1 Inclusion Rule

A prompt is included when changing or omitting it could change a reported metric, model comparison, benchmark construction step, annotation protocol, or reproducibility claim. For each included prompt, the experiment records the prompt text, model identifier, provider, date, key decoding parameters, input modalities, output format, random seed if applicable, and source result file.

A.2 Track A Pairwise Judgement Prompts

A.2.1 TA-PAIR-ZS-v1: Zero-shot Pairwise Visual Design Prompt

Purpose: Baseline pairwise GUI screenshot judgement for Track A, including the order-swapped reliability condition. The model receives two screenshots labelled Image A and Image B and returns only the preferred letter.

Metadata:

- **Track / task:** Track A, pairwise preference judgement
- **Prompt ID:** TA-PAIR-ZS-v1

- **Models used:** gpt-4o (OpenAI); claude-sonnet-4-5-20250929 (chatanywhere-anthropic proxy)
- **Input modalities:** two PNG screenshots (Image A, Image B) + user-role text
- **Message structure:** single user message; no system message
- **Decoding parameters:** temperature = 0, max_tokens = 5
- **Seed:** 42 (pair sampling); model temperature is 0 so output is deterministic
- **Order-swap:** applied; each pair is evaluated in both AB and BA orders
- **Dataset:** biglab/uiclip_human_data_hf (HuggingFace), train split
- **Source:** scripts/track_a_eval.py, lines 44–52 (ZEROSHOT_PROMPT)
- **First use:** 2026-05-06 (gpt-4o pilot); 2026-05-19 (claude-sonnet-4-5 run)
- **Result files:** scripts/results/track_a_eval_gpt-4o_20260506_*;
scripts/results/track_a_eval_chatanywhere-anthropic_claude-sonnet-4-5-20250929_*

You are evaluating two user interface screenshots for visual design quality.

Image A is shown first, Image B is shown second.

Which interface has better visual design overall? Consider layout, visual hierarchy, clarity, and aesthetic quality.

Reply with ONLY the letter A or B. Nothing else.

A.2.2 TA-PAIR-RUBRIC-v1: Rubric-guided Pairwise Visual Design Prompt

Purpose: Rubric-guided Track A pairwise GUI screenshot judgement. Adds four explicit design dimensions before requiring the same A/B output format as the zero-shot condition. Used in the prompting-strategy ablation.

Metadata:

- **Track / task:** Track A, pairwise preference judgement (rubric condition)
- **Prompt ID:** TA-PAIR-RUBRIC-v1
- **Models used:** gpt-4o (OpenAI)
- **Input modalities:** two PNG screenshots (Image A, Image B) + user-role text
- **Message structure:** single user message; no system message

- **Decoding parameters:** temperature = 0, max_tokens = 5
- **Seed:** 42 (pair sampling)
- **Order-swap:** applied
- **Dataset:** biglab/uiclip_human_data_hf (HuggingFace), train split
- **Source:** scripts/track_a_eval.py, lines 54–68 (RUBRIC_PROMPT)
- **First use:** 2026-05-19
- **Result files:** scripts/results/track_a_eval_gpt-4o_20260506_142748_pair_review/

You are an expert UI design reviewer evaluating two user interface screenshots.
Image A is shown first, Image B is shown second.

Score each interface on the following four dimensions before deciding:

1. Visual structure - layout, alignment, spacing, and visual hierarchy.
2. Information clarity - readability, labelling, and grouping of related items.
3. Aesthetic quality - typography, colour use, and overall polish.
4. Inferred usability - how easy the interface looks to use at a glance.

Briefly compare the two interfaces dimension by dimension in your head, then
pick the one that is better overall.

Reply with ONLY the letter A or B on the last line. Nothing else.

A.3 Track B GUI Generation Prompts

A.3.1 Track B Generation Prompt Version History

Track B generation prompts changed repeatedly during smoke testing. To keep the appendix focused on reproducibility rather than serving as a full prompt archive, this appendix reproduces in full the frozen baseline prompt that produces the reported Track B pilot and leaderboard artifacts, TB-GEN-v9, the later F10 diagnostic prompts TB-GEN-v14 and TB-GEN-v15, and the compact-input candidate TB-GEN-v16. All earlier smoke-test variants (TB-GEN-v1 through v8, v12, and v13) were diagnostic iterations that did not contribute reported metrics; they are summarized in Table A.1 and remain recoverable in full from the source template in `scripts/generate_track_b_ui.py`, per-run `generation_metadata.json` files, and the revision log when needed to interpret a specific pilot artifact.

A.3.2 TB-GEN-v9: Per-Item Workflow-Label Generation Prompt

Purpose: Practical Track B baseline prompt for the cross-item smoke batch. This version generalizes the v8 workflow-control idea by filling the workflow label list automatically from each benchmark item’s workflow file rather than using a hand-written item-specific mapping.

Metadata:

- **Track / task:** Track B, GUI generation
- **Prompt ID:** TB-GEN-v9
- **Version change:** revises TB-GEN-v8 by replacing the hard-coded F09 workflow-control list with an automatically extracted `{workflow_labels}` slot
- **Models used:** qwen3.6-35b-a3b via GWDG/SAIA OpenAI-compatible API
- **Input modalities:** normalized requirement text, Vision2Web workflow JSON, local resource-path list, route contract, and prototype screenshots
- **Message structure:** system message plus one user message containing text and prototype images
- **Decoding parameters:** temperature = 0.2, max_tokens = 20000 in smoke tests
- **Dataset:** zai-org/Vision2Web, reduced Level 1 / Level 2 pilot subset
- **Source:** scripts/generate_track_b_ui.py
- **First use:** 2026-05-26, Europe/Berlin
- **Result files:** data/track_b/items/F01_1daycloud/generated/gwdg_qwen36_35b_v9_smoke/;
data/track_b/items/F03_about_gitlab/generated/gwdg_qwen36_35b_v9_smoke/;
data/track_b/items/F10_gourmania/generated/gwdg_qwen36_35b_v9_smoke/

System:

You are a senior frontend engineer building a benchmark submission for a GUI evaluation study.
Create a polished, working website implementation that can be opened directly from a local index.html file.

User:

Prompt ID: TB-GEN-v9

Build a minimal complete single-file HTML implementation for the Track B GUI item below.

Hard requirements:

- Return exactly one complete HTML document ending with `</html>`.
- Include inline CSS and JavaScript only. No build step, backend, network access, external CDN, or markdown.
- Local assets must be referenced as `../../resources/<subpath>`.
- Use the prototype screenshots for overall structure and visual cues, but produce a minimal benchmark implementation.

- Primary viewport is desktop: 1365px and 1920px. Mobile quality is not a hard gate.
- Implement every route in the route contract as a <section id="..." data-track-route>.
- Include window.__TRACK_B_ROUTES exactly as given and a showPage(routeId) handler before </body>.

Mandatory workflow controls:

- The following exact workflow click labels were extracted from the workflow checks. Every label below must appear as visible in the workflow.
- Never use <div>, , , or other inert elements for workflow-required click labels.
- Every workflow control must include data-route-target="<route_id>" and onclick="showPage('<route_id>')".
- If a click target is a feature phrase such as "GitLab Duo", use that feature phrase as the visible label.
- Choose the route target from the route contract that best matches the label and workflow validation.
- Secondary navigation items may be visible non-functional placeholders only if they are not listed in the extracted workflow.

Strict compactness limits:

- Use one shared header and one shared footer only.
- For each route, use at most one h1, one short paragraph, one compact card/list, and one optional tiny table.
- Use at most 3 cards per route, 3 bullets per list, and 3 rows per table.
- Do not copy long requirement text. Use short labels and short summaries.
- Avoid decorative animations, large CSS frameworks, inline SVG art, exhaustive menus, and long repeated sections.
- If output budget feels tight, remove visual detail first. Never omit routes, workflow controls, JavaScript, or closing tags.

Route contract:

```
{route_contract}
```

Required JavaScript:

```
<script>
window.__TRACK_B_ROUTES = {route_ids_json};
function showPage(routeId) {
  if (!window.__TRACK_B_ROUTES.includes(routeId)) return;
  document.querySelectorAll('[data-track-route]').forEach(function(section) {
    section.hidden = section.id !== routeId;
  });
  document.querySelectorAll('[data-route-target]').forEach(function(link) {
    link.setAttribute('aria-current', link.dataset.routeTarget === routeId ? 'page' : 'false');
  });
  if (history.replaceState) history.replaceState(null, '', '#' + routeId);
  setTimeout(function() { window.scrollTo(0, 0); }, 0);
}
document.addEventListener('DOMContentLoaded', function() {
  var initial = location.hash ? location.hash.slice(1) : window.__TRACK_B_ROUTES[0];
  showPage(window.__TRACK_B_ROUTES.includes(initial) ? initial : window.__TRACK_B_ROUTES[0]);
});
</script>
```

Item ID: {item_id}

Source task: {source_task_name}

Source level: {source_level}

Use for dynamic validation: {use_for_dynamic}

Normalized requirement:

```
{requirement}
```

Workflow checks:

```
{workflow}
```

Available local resource paths:

```
{resource_paths}
```

Prototype screenshots are attached after this text in this order:

{prototype_list}

A.3.3 TB-GEN-v14: Visual-Grounded Workflow-Control Diagnostic Prompt

Purpose: F10 diagnostic prompt introduced after TB-GEN-v9 and cheap second-model smoke tests produced bare or weakly interactive pages. This version tests whether stronger visual-grounding instructions and an explicit machine-checkable workflow-control contract improve generated artifacts without changing the underlying F10 benchmark item.

Metadata:

- **Track / task:** Track B, GUI generation
- **Prompt ID:** TB-GEN-v14
- **Version change:** revises TB-GEN-v9/v12 by removing strong compactness limits, requiring local images, and clarifying that workflow context phrases are not necessarily clickable controls
- **Model used:** qwen3-omni-30b-a3b-instruct via GWDG/SAIA OpenAI-compatible API
- **Input modalities:** normalized requirement text, Vision2Web workflow JSON, local resource-path list, route contract, and prototype screenshots
- **Message structure:** system message plus one user message containing text and prototype images
- **Decoding parameters:** temperature = 0.2, max_tokens = 20000
- **First use:** 2026-06-04, Europe/Berlin
- **Source:** scripts/generate_track_b_ui.py
- **Result file:** data/track_b/items/F10_gourmania/generated/gwdg_qwen3_omni_30b_v14_f

System:

You are a senior frontend engineer building a benchmark submission for a GUI evaluation study.
Create a polished, working website implementation that can be opened directly from a local index.html file.

User:

Prompt ID: TB-GEN-v14

Build a visually grounded, complete single-file HTML implementation for the Track B GUI item below.

Hard requirements:

- Return exactly one complete HTML document ending with </html>.

- Include inline CSS and JavaScript only. No build step, backend, network access, external CDN, or markdown.
- Local assets must be referenced exactly as ../../resources/<subpath> using paths from the resource list.
- Implement every route in the route contract as a <section id="..." data-track-route>.
- Include window.__TRACK_B_ROUTES exactly as given and a showPage(routeId) handler before </body>.
- The first route in window.__TRACK_B_ROUTES must be the default page shown on initial load.

Visual-grounding requirements:

- Use the prototype screenshots for layout, hierarchy, colors, spacing, and visible content priorities.
- Use local image assets whenever they are available. Do not replace provided images with plain text boxes, CSS placeholders.
- The homepage must include visible elements when image assets exist.
- Product, recipe, article, service, and video card/list sections should use relevant elements when matching or placeholder.
- Select images by filename and requirement context. For example, filenames that resemble a recipe, cookbook, logo, video, or product.
- If no exact image match is obvious, choose the closest local image rather than omitting imagery.
- Reduce prose before removing images, workflow controls, route sections, or JavaScript.

Required workflow-control contract:

- The controls below are machine-checkable requirements. For each item, create an <a> or <button> whose visible text exactly matches the visible_text value.
- The listed visible_text values are the actual click targets. Context phrases in workflow actions, such as section names or product names, are not click targets.
- Do not replace visible_text with a synonym, longer phrase, icon-only label, or nearby non-clickable heading.
- Do not implement workflow-required controls as <div>, , , image-only controls, or any other inert element, even if it is visually prominent.
- If a workflow target is visually a card or image tile, wrap the card content in an <a> or <button> and keep the requirement text visible.
- The required_element example is the safest minimal implementation; copy its text, data-route-target, and onclick route_id into the workflow_controls object.

Implementation guidance:

- Build a recognizable desktop page, not a bare wireframe.
- Prefer a shared header/navigation and shared footer, with route-specific visual content.
- Include enough cards/tables/lists to satisfy the written requirement and workflow validations, but do not reproduce exact content.
- Use semantic HTML, visible focus states, alt text for images, and accessible <a>/<button> controls.
- Secondary navigation items may be visible non-functional placeholders only if they are not workflow-required controls.

Route contract:

```
{route_contract}
```

Required JavaScript:

```
<script>
window.__TRACK_B_ROUTES = {route_ids_json};
function showPage(routeId) {
  if (!window.__TRACK_B_ROUTES.includes(routeId)) return;
  document.querySelectorAll('[data-track-route]').forEach(function(section) {
    section.hidden = section.id !== routeId;
  });
  document.querySelectorAll('[data-route-target]').forEach(function(link) {
    link.setAttribute('aria-current', link.dataset.routeTarget === routeId ? 'page' : 'false');
  });
  if (history.replaceState) history.replaceState(null, '', '#' + routeId);
  setTimeout(function() { window.scrollTo(0, 0); }, 0);
}
document.addEventListener('DOMContentLoaded', function() {
  var initial = location.hash ? location.hash.slice(1) : window.__TRACK_B_ROUTES[0];
  showPage(window.__TRACK_B_ROUTES.includes(initial) ? initial : window.__TRACK_B_ROUTES[0]);
});
</script>
```

Item ID: {item_id}

Source task: {source_task_name}

Source level: {source_level}

Use for dynamic validation: {use_for_dynamic}

Normalized requirement:
{requirement}

Workflow checks:
{workflow}

Available local resource paths:
{resource_paths}

Prototype screenshots are attached after this text in this order:
{prototype_list}

A.3.4 TB-GEN-v15: Stricter Route and Click-Target Diagnostic Prompt

Purpose: F10 diagnostic prompt introduced after TB-GEN-v14 improved image usage but still failed static gate checks by using non-contract `showPage()` route IDs and replacing the required `Gefilte Fish` click target with a generic “View Recipe” link.

Metadata:

- **Track / task:** Track B, GUI generation
- **Prompt ID:** TB-GEN-v15
- **Version change:** revises TB-GEN-v14 by forbidding non-contract `showPage()` route IDs and by requiring the clickable element’s own visible text to exactly match each `visible_text`
- **Model used:** `qwen3-omni-30b-a3b-instruct` via GWDG/SAIA OpenAI-compatible API
- **Input modalities:** normalized requirement text, Vision2Web workflow JSON, local resource-path list, route contract, and prototype screenshots
- **Message structure:** system message plus one user message containing text and prototype images
- **Decoding parameters:** `temperature = 0.2`, `max_tokens = 20000`
- **First use:** 2026-06-04, Europe/Berlin
- **Source:** `scripts/generate_track_b_ui.py`
- **Result file:** `data/track_b/items/F10_gourmania/generated/gwdg_qwen3_omni_30b_v15_f`

System:

You are a senior frontend engineer building a benchmark submission for a GUI evaluation study.
Create a polished, working website implementation that can be opened directly from a local `index.html` file.

User:

Prompt ID: TB-GEN-v15

Build a visually grounded, complete single-file HTML implementation for the Track B GUI item below.

Hard requirements:

- Return exactly one complete HTML document ending with `</html>`.
- Include inline CSS and JavaScript only. No build step, backend, network access, external CDN, or markdown.
- Local assets must be referenced exactly as `../resources/<subpath>` using paths from the resource list.
- Implement every route in the route contract as a `<section id="..." data-track-route>`.
- Include `window.__TRACK_B_ROUTES` exactly as given and a `showPage(routeId)` handler before `</body>`.
- Never call `showPage()` with a route id that is not listed in `window.__TRACK_B_ROUTES`.
- Secondary menu items that do not have a route in the route contract must be plain placeholders without onclick, without

Visual-grounding requirements:

- Use the prototype screenshots for layout, hierarchy, colors, spacing, and visible content priorities.
- Use local image assets whenever they are available. Do not replace provided images with plain text boxes, CSS placeholders.
- The homepage must include visible `<img>` elements when image assets exist.
- Product, recipe, article, service, and video card/list sections should use relevant `<img>` elements when matching or placeholder.
- Select images by filename and requirement context. If no exact image match is obvious, choose the closest local image match.
- Reduce prose before removing images, workflow controls, route sections, or JavaScript.

Required workflow-control contract:

- The controls below are machine-checkable requirements. For each item, create an `<a>` or `<button>` whose visible text exactly matches the required `visible_text`.
 - Copy each `required_element` line into the HTML output exactly once or adapt only the tag name from `<a>` to `<button>`. Keep the `required_element` text exactly as is.
 - The required clickable element's own visible text must be exactly `visible_text`. Do not use "View", "View Recipe", "Read More", etc.
 - The listed `visible_text` values are the actual click targets. Context phrases in workflow actions, such as section names, are not click targets.
 - Do not implement workflow-required controls as `<div>`, `<span>`, `<li>`, image-only controls, or any other inert element, even if it is visually a card or image tile.
 - If a workflow target is visually a card or image tile, put the required `<a>` or `<button>` inside that card, and make the visible text match the `visible_text`.
 - Do not hide required controls in comments, scripts, templates, off-screen text, `display:none`, `visibility:hidden`, or `aria-hidden=true`.
- {workflow_controls}

Implementation guidance:

- Build a recognizable desktop page, not a bare wireframe.
- Prefer a shared header/navigation and shared footer, with route-specific visual content.
- Include enough cards/tables/lists to satisfy the written requirement and workflow validations, but do not reproduce extraneous content.
- Use semantic HTML, visible focus states, alt text for images, and accessible `<a>/<button>` controls.
- Before returning the HTML, self-check these conditions:
 1. Every `required_element visible_text` appears as visible text inside an `<a>` or `<button>`.
 2. Every `showPage('...')` route is present in `window.__TRACK_B_ROUTES` and has a matching section id.
 3. The page uses local `<img>` assets when image assets were provided.

Route contract:

{route_contract}

Required JavaScript:

```
<script>
window.__TRACK_B_ROUTES = {route_ids_json};
function showPage(routeId) {
  if (!window.__TRACK_B_ROUTES.includes(routeId)) return;
  document.querySelectorAll('[data-track-route]').forEach(function(section) {
    section.hidden = section.id !== routeId;
  });
  document.querySelectorAll('[data-route-target]').forEach(function(link) {
    link.setAttribute('aria-current', link.dataset.routeTarget === routeId ? 'page' : 'false');
  });
  if (history.replaceState) history.replaceState(null, '', '#' + routeId);
  setTimeout(function() { window.scrollTo(0, 0); }, 0);
}
```

```
document.addEventListener('DOMContentLoaded', function() {
  var initial = location.hash ? location.hash.slice(1) : window.__TRACK_B_ROUTES[0];
  showPage(window.__TRACK_B_ROUTES.includes(initial) ? initial : window.__TRACK_B_ROUTES[0]);
});
</script>
```

```
Item ID: {item_id}
Source task: {source_task_name}
Source level: {source_level}
Use for dynamic validation: {use_for_dynamic}
```

```
Normalized requirement:
{requirement}
```

```
Workflow checks:
{workflow}
```

```
Available local resource paths:
{resource_paths}
```

```
Prototype screenshots are attached after this text in this order:
{prototype_list}
```

A.3.5 TB-GEN-v16: Compact-Input Visual Workflow Candidate

Purpose: Candidate prompt introduced after the TB-GEN-v15 small-batch experiment showed that v15 could pass F10 but did not generalize to larger items because of context/output pressure, repeated workflow input, and exact text treatment of semantic visual actions such as logo clicks.

Metadata:

- **Track / task:** Track B, GUI generation
- **Prompt ID:** TB-GEN-v16
- **Version change:** revises TB-GEN-v15 by using a compact route/evidence workflow summary instead of the full workflow JSON, separating exact visible-text controls from semantic/visual targets, adding bounded generation limits, and pairing the prompt with deterministic input packing (`compact_route_evidence_v1`, ranked resource paths, and prototype image downscaling for compact runs)
- **Model intended for first smoke run:** qwen3.6-35b-a3b via GWDG/SAIA OpenAI-compatible API
- **Input modalities:** normalized requirement text, compact workflow route/evidence summary, ranked local resource-path list, route contract, and prototype screenshots

- **Message structure:** system message plus one user message containing text and prototype images
- **Decoding parameters:** planned smoke-test default temperature = 0.2; max_tokens chosen per model capability table rather than as a quality score
- **First implementation:** 2026-06-05, Europe/Berlin
- **Source:** scripts/generate_track_b_ui.py
- **Result file:** not yet frozen; future smoke runs must record the generated directory and generation_metadata.json

System:

You are a senior frontend engineer building a benchmark submission for a GUI evaluation study.
Create a polished, working website implementation that can be opened directly from a local index.html file.

User:

Prompt ID: TB-GEN-v16

Build a visually grounded, complete single-file HTML implementation for the Track B GUI item below.

Hard requirements:

- Return exactly one complete HTML document ending with </html>.
- Include inline CSS and JavaScript only. No build step, backend, network access, external CDN, or markdown.
- Local assets must be referenced exactly as ../../resources/<subpath> using paths from the resource list.
- Implement every route in the route contract as a <section id="..." data-track-route>.
- Include window.__TRACK_B_ROUTES exactly as given and a showPage(routeId) handler before </body>.
- Never call showPage() with a route id that is not listed in window.__TRACK_B_ROUTES.
- Secondary menu items that do not have a route in the route contract must be plain placeholders without onclick, without

Visual-grounding requirements:

- Use the prototype screenshots for layout, hierarchy, colors, spacing, and visible content priorities.
- Use local image assets whenever they are available. Do not replace provided images with plain text boxes, CSS placeholders.
- Use a bounded number of images: include at least one relevant local image on visual routes when assets exist, but avoid
- Product, recipe, article, service, and video card/list sections should use relevant elements when matching or pla
- Select images by filename and requirement context. If no exact image match is obvious, choose the closest local image f
- Reduce repeated prose before removing images, workflow controls, route sections, validation evidence, or JavaScript.

Required exact workflow-control contract:

- The controls below are machine-checkable exact-text requirements. For each item, create an <a> or <button> whose visible
 - Copy each required_element line into the HTML output exactly once or adapt only the tag name from <a> to <button>. Keep
 - The required clickable element's own visible text must be exactly visible_text. Do not use "View", "View Recipe", "Read
 - Do not implement workflow-required exact-text controls as <div>, , , image-only controls, or any other inert
 - Semantic/visual targets listed in the compact workflow summary, such as a logo click, must be clickable and route corre
- {workflow_controls}

Bounded implementation guidance:

- Build a recognizable desktop page, not a bare wireframe.
- Prefer one shared header/navigation and one shared footer, with route-specific visual content.
- For each route, include one clear heading, enough validation evidence text, and at most one compact table/list plus a s
- Use at most 4 cards per route, 4 rows per table, and 4 bullets per list unless a validation explicitly requires more.
- Use semantic HTML, visible focus states, alt text for images, and accessible <a>/<button> controls.
- Before returning the HTML, self-check these conditions:
 1. Every exact required_element visible_text appears as visible text inside an <a> or <button>.
 2. Every semantic/visual workflow target routes correctly without requiring its descriptive phrase as visible text.

3. Every `showPage(...)` route is present in `window.__TRACK_B_ROUTES` and has a matching section id.
4. Every route includes the evidence needed by the compact workflow validations.
5. The page uses local `<img>` assets when image assets were provided.

Route contract:
`{route_contract}`

Required JavaScript:
`<script>`
`window.__TRACK_B_ROUTES = {route_ids_json};`
`function showPage(routeId) {`
 `if (!window.__TRACK_B_ROUTES.includes(routeId)) return;`
 `document.querySelectorAll('[data-track-route]').forEach(function(section) {`
 `section.hidden = section.id !== routeId;`
 `});`
 `document.querySelectorAll('[data-route-target]').forEach(function(link) {`
 `link.setAttribute('aria-current', link.dataset.routeTarget === routeId ? 'page' : 'false');`
 `});`
 `if (history.replaceState) history.replaceState(null, '', '#' + routeId);`
 `setTimeout(function() { window.scrollTo(0, 0); }, 0);`
`}`
`document.addEventListener('DOMContentLoaded', function() {`
 `var initial = location.hash ? location.hash.slice(1) : window.__TRACK_B_ROUTES[0];`
 `showPage(window.__TRACK_B_ROUTES.includes(initial) ? initial : window.__TRACK_B_ROUTES[0]);`
`});`
`</script>`

Item ID: `{item_id}`
 Source task: `{source_task_name}`
 Source level: `{source_level}`
 Use for dynamic validation: `{use_for_dynamic}`

Normalized requirement:
`{requirement}`

Compact workflow checks, deduplicated from `workflow.json`:
`{workflow}`

Relevant local resource paths:
`{resource_paths}`

Prototype screenshots are attached after this text in this order:
`{prototype_list}`

A.4 Track B Static-Visual Judge and Human Review Prompts

A.4.1 LB-JUDGE-v1: Base Static-Visual Checklist Prompt

Purpose: Static visual scoring for standardized Track B page screenshots. The prompt asks a multimodal LLM to answer the same 16 binary checklist items used by the human review interface, returning machine-readable JSON.

Metadata:

- **Track / task:** Track B, static visual judging
- **Prompt ID:** LB-JUDGE-v1-base
- **Models used in development comparisons:** internvl3.5-30b-a3b, qwen3-omni-30b-a3b-instruct, gemma-4-31b-it, gpt-4o-mini, and gpt-4.1-mini, depending on provider availability
- **Input modalities:** one standardized PNG screenshot plus text checklist
- **Message structure:** system message plus one user message with text and image
- **Decoding parameters:** temperature = 0.0, max_tokens = 800
- **Source:** scripts/run_visual_judge.py
- **Result files:** data/track_b/visual_human_review/llm_judge_*.json and human_vs_llm_comparison*.json

System:

You are a careful visual UI reviewer. You see ONE screenshot of ONE page of a web interface. Judge ONLY what is visible. Do not guess about interactivity or pages you cannot see. Answer each checklist item strictly true or false. Return ONLY valid JSON, no prose.

User:

Answer every checklist item for this screenshot (true = page satisfies it).

Layout & Visual Hierarchy

- L1: There is a clear primary element or focal point.
- L2: Elements are free of overlap or obvious misalignment.
- L3: Alignment follows a consistent grid/structure.
- L4: Spacing and padding are consistent across the page.

Information Organization & Clarity

- O1: Related content is grouped into clear sections.
- O2: The page is uncluttered, not visually overwhelming.
- O3: Sections are labelled or easy to tell apart.
- O4: The main purpose/content is clear at a glance.

Typography & Readability

- T1: Clear text hierarchy (headings vs body distinguishable).
- T2: Body text is a comfortably readable size.
- T3: Line length / text width is reasonable.
- T4: Font usage is consistent (no clashing typefaces).

Visual Consistency

- C1: Buttons and controls share a consistent style.
- C2: Colour palette is coherent (no clashing colours).
- C3: Navigation / header styling is consistent.
- C4: Repeated components look uniform.

Return JSON exactly:

```
{"answers": {"L1": true, "L2": true, "L3": true, "L4": true,
  "O1": true, "O2": true, "O3": true, "O4": true,
  "T1": true, "T2": true, "T3": true, "T4": true,
  "C1": true, "C2": true, "C3": true, "C4": true}, "confidence": 0.0}
```

A.4.2 LB-JUDGE-v1: Strict Variant

Purpose: Bias-reduction variant introduced after base prompts across several vision models produced all-true or near-all-true outputs. The checklist items are identical to LB-JUDGE-v1-base; only the system instruction and the lead-in before the JSON schema change.

Metadata:

- **Track / task:** Track B, static visual judging
- **Prompt ID:** LB-JUDGE-v1-strict
- **Version change:** adds fault-finding instruction and changes “Return JSON exactly” to “Look for problems first. Return JSON exactly”
- **Source:** scripts/run_visual_judge.py

System:

You are a STRICT, critical visual UI reviewer evaluating an AI-generated web page. Most AI-generated pages contain at least some real visual flaws (inconsistent spacing, weak contrast, misalignment, cramped or oversized text, clashing colours, uneven components). Your job is to FIND those flaws, not to approve. Mark an item true ONLY if the criterion is clearly and fully satisfied with no visible issue. If there is ANY visible problem, or you are not clearly confident, mark it false. A page that is true on every item is rare; before answering, identify the single worst visual problem on the page and make sure the relevant item is false. Return ONLY valid JSON, no prose.

User:

Same 16-item checklist as LB-JUDGE-v1-base, with this lead-in before the schema:

Look for problems first. Return JSON exactly:

A.4.3 LB-JUDGE-v1: Strict Few-Shot Variant

Purpose: Calibration variant for static visual scoring. It uses the strict system and user prompts above, then prepends three human-labelled example screenshots before the scored screenshot. The example pages are excluded from evaluation when computing human-vs-LLM agreement.

Metadata:

- **Track / task:** Track B, static visual judging with calibration examples
- **Prompt ID:** LB-JUDGE-v1-strict-fewshot
- **Example pages:** F10_gourmania::01_homepage (human page score 0.50), F01_1daycloud::04_contact (0.75), and F01_1daycloud::02_academy (1.00)

- **Message structure:** strict system message; for each example, one user image+checklist message followed by one assistant JSON answer; then the scored user image+checklist message
- **Source:** scripts/run_visual_judge.py

System:

Use the LB-JUDGE-v1-strict system message.

Few-shot messages:

For each calibration page, send the same LB-JUDGE-v1-strict user checklist with the example screenshot, followed by an assistant message:

```
{"answers": {"L1": <human label>, ..., "C4": <human label>},
  "confidence": 1.0}
```

Scored message:

Send the same LB-JUDGE-v1-strict user checklist with the target screenshot.

A.4.4 HUMAN-VISUAL-v1: Human Review Checklist

Purpose: Human annotation protocol for validating the static visual checklist against LLM judges. The self-contained review page shows one standardized screenshot at a time and asks the rater to answer the same 16 binary checklist items as LB-JUDGE-v1.

Metadata:

- **Track / task:** Track B, human static-visual annotation
- **Prompt ID:** HUMAN-VISUAL-v1
- **Interface source:** scripts/build_visual_human_review.py
- **Original export:** data/track_b/visual_human_review/visual_human_review.json
- **Extended export:** data/track_b/visual_human_review/visual_human_review_extended
- **Extended setting:** deterministic shuffled order, run names hidden from the page, separate localStorage key and export name

Static-Visual Human Review

For each screenshot, answer the same 16 checklist items:

L1-L4 Layout & Visual Hierarchy
 O1-O4 Information Organization & Clarity
 T1-T4 Typography & Readability
 C1-C4 Visual Consistency

Each item is answered Yes or No. The interface also provides All Yes and All No buttons for a page; the rater can then correct individual items before exporting JSON.

A.5 Prompt Slots for Later Experiments

Future thesis-impacting prompts should be added here before or immediately after their first experimental use. Expected prompt families include Track B static rubric scoring prompts, LLM-based dynamic validation prompts if such validators are later used, output-schema prompts, and human annotation instructions.

Prompt ID	Use	Main change	Status in appendix
TB-GEN-v1	First smoke run	Baseline single-file HTML generation prompt for normalized Vision2Web pilot items.	Summarized only
TB-GEN-v2	Smoke iteration	Added explicit completion, same-file JavaScript handlers, workflow-action non-placeholder requirements, and responsive-layout constraints.	Summarized only
TB-GEN-v3-v4	Intermediate iteration	Refined workflow route handling before the fixed route-contract version.	Summarized only
TB-GEN-v5	Route-contract smoke run	Added a fixed route contract, a minimal route handler, no-scroll hash replacement, and stricter mobile overflow safeguards.	Summarized only
TB-GEN-v6	Reproducible smoke template	Made desktop workflow fidelity the hard generation target and clarified handling of unavailable downloadable assets.	Summarized only
TB-GEN-v7	Compact same-prompt comparison template	Keeps the same route/workflow contract as v6 but instructs models to produce a shorter complete implementation to reduce truncation and gateway-timeout risk.	Summarized only
TB-GEN-v8	Compact gated comparison template	Adds explicit exact workflow-label-to-route mappings and forbids inert elements for workflow-required controls after v7 produced an inert Local Forms control.	Summarized only
TB-GEN-v9	Per-item workflow-label prompt (frozen baseline)	Generalizes v8 by automatically extracting workflow click labels from each item’s workflow file instead of hard-coding the F09 label-to-route list. Frozen as the practical baseline for the cross-item pilot batch.	Full text below
TB-GEN-v12	Machine-checkable workflow-control contract	Adds per-item <code>visible_text</code> , <code>route_id</code> , and <code>required_element</code> entries to reduce ambiguity around exact clickable labels; did not resolve the exact-label case and was not adopted.	Summarized only
TB-GEN-v13	Required workflow-control navigation block	Replaces per-control examples with a complete navigation block to copy into the page header; both smoke attempts failed at the provider layer (HTTP 500) and produced no artifact.	Summarized only
TB-GEN-v14	F10 visual-grounding diagnostic	Removes the strong compactness bias, requires local images, and uses per-control <code>required_element</code> entries while clarifying that workflow context phrases are not necessarily clickable controls.	Full text below
TB-GEN-v15	F10 control-contract diagnostic	Tightens v14 after F10 v14 failed: forbids non-contract <code>showPage()</code> routes and forbids replacing required control text with labels such as “View Recipe”.	Full text below
TB-GEN-v16	Compact-input candidate	Keeps v15’s exact-control contract but uses a compact route/evidence workflow summary, separates semantic visual targets from exact visible text controls, adds bounded generation, and uses deterministic ranked resources plus prototype-image downscaling in the generator input policy.	Full text below

Table A.1: Summary of Track B GUI generation prompt versions. Only the current templates are reproduced in full in this appendix; prior versions are retained through experiment artifacts and source history.